SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	**K.Siva**	**Reg. No.:**	**192525461**
Date:	27.10.26	**Duration:**	60 minutes

Code of Conduct

I K.Siva / 192525461 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ***K.Siva***

Scenario-Based Requirement Analysis and SRS

Project: Smart Pharmacy Inventory and Prescription Management System | **Course:** CSA10 / CSA1004 Software Engineering | **CO2 - Assessment Tool 1**

1. Study and Analysis of the Scenario

1.1 Business Domain

Healthcare retail: community pharmacy operations covering medicine inventory control, regulated dispensing of prescription medicines, supplier procurement and sales. The domain is time-sensitive and safety-related because expired or wrongly dispensed medicines can harm patients.

1.2 Scenario Summary

A pharmacy holds hundreds of medicines, and each medicine may have several batches on the shelf, each with its own batch number, quantity and expiry date. Pharmacy staff currently track this on paper registers or spreadsheets. The pharmacy needs a system that manages medicine inventory, prescriptions, suppliers, expiry dates and sales transactions, and that generates automated stock alerts, while the pharmacist keeps every professional decision.

1.3 Existing Challenges

- Batches expire unnoticed at the back of a shelf, causing wastage.
- Fast-moving medicines run out before they are reordered; shortages are found only when a customer asks.
- Prescription-only medicines may be sold without the prescription being recorded.
- Paper registers are slow to search and often omit batch numbers or expiry dates.
- Manual stock counts give only occasional snapshots, and owner reports are slow and error-prone.

1.4 System Objectives

- Maintain medicine inventory with quantities, units and reorder levels; track batch numbers and expiry dates.
- Manage prescriptions, supplier information and purchase orders; track purchases, sales, returns and write-offs.
- Generate low-stock alerts, identify near-expiry and expired products, and generate inventory reports.
- Expected outcomes: reduced wastage, improved inventory accuracy, fewer shortages, better expiry management, faster operations and improved record management.

1.5 Stakeholders

- **Pharmacist** - checks prescriptions, dispenses, makes final decisions. *Main need:* accurate stock and batch data under time pressure.
- **Pharmacy assistant** - receives stock, processes sales, shelves medicines. *Main need:* fast and correct data entry in busy periods.
- **Owner / manager** - approves purchases, sets thresholds, reviews reports. *Main need:* lower expired-stock losses; reliable demand view.
- **Supplier** - delivers medicines with batch and expiry details. *Main need:* matching orders and timely deliveries.
- **Prescriber (doctor)** - issues prescriptions that the pharmacy records. *Main need:* complete, readable prescription details.
- **Patient** - receives medicines and counselling. *Main need:* availability, short waiting time, privacy.
- **System administrator** - manages accounts, roles and backups. *Main need:* protecting private records; avoiding downtime.

2. Functional Requirements

Functional requirements describe what the system must do. Priority uses MoSCoW (M = Must, S = Should).

- **FR1** (user management, Must): The system shall provide secure login and role-based access for pharmacist, assistant, manager and administrator.
- **FR2** (user management, Must): The administrator shall create, edit and deactivate staff accounts and assign roles.
- **FR3** (medicine and batch, Must): The system shall register medicines with name, category, unit, storage details and reorder level.
- **FR4** (medicine and batch, Must): The system shall store each batch with batch number, expiry date, quantity, supplier and purchase date.
- **FR5** (medicine and batch, Must): The system shall search medicines by name or barcode and show all their batches with quantities and expiry dates.
- **FR6** (prescription, Must): The system shall record prescriptions with patient, prescriber, date and prescribed items.
- **FR7** (prescription, Must): The system shall flag prescription-only medicines and require a linked prescription before dispensing.
- **FR8** (sales and transactions, Must): The system shall record sales, purchases, returns and write-offs with date, time, user, quantity and batch, and update stock automatically.
- **FR9** (sales and transactions, Must): The system shall suggest the earliest-expiring valid batch (FEFO) and block the sale of expired batches or of more than the available stock.
- **FR10** (supplier, Must): The system shall maintain supplier details and the medicines each supplier provides.
- **FR11** (supplier, Should): The system shall create purchase orders and link them to received stock.
- **FR12** (alerts, Must): The system shall raise a low-stock alert naming the medicine and usual supplier when total valid quantity reaches the reorder level.
- **FR13** (alerts, Must): The system shall mark batches near expiry (configurable period) and expired, and exclude expired batches from available stock.
- **FR14** (alerts, Should): Authorised managers shall configure reorder levels and the near-expiry warning period per medicine.
- **FR15** (reports, Should): The system shall export inventory, sales and expiry reports for a chosen period as PDF or CSV.
- **FR16** (security and audit, Must): The system shall keep an audit log of who changed what and when, and reject negative or impossible quantities.

Note: the system records, validates and raises alerts; it does not judge whether a prescription is clinically appropriate. That decision stays with the licensed pharmacist.

3. Non-Functional Requirements

- **NFR1 Performance:** Search a medicine and display its batches within 3 seconds; show a triggered alert within 60 seconds of a saved change. *Verified by:* timed prototype test.
- **NFR2 Security:** Hashed passwords, role-based permissions, HTTPS, and an audit log; only pharmacists approve Rx-only sales; only managers change thresholds. *Verified by:* access tests by role.
- **NFR3 Reliability:** Validate every entry, store exactly one record per submitted transaction, keep transaction history and back up regularly. *Verified by:* duplicate-submit test; backup restore.
- **NFR4 Usability:** Simple screens, clear units, readable alert messages (e.g. "Batch B1204 expires in 30 days"), search by name or barcode. *Verified by:* staff feedback each sprint.
- **NFR5 Scalability:** Every record carries pharmacy, medicine and batch identifiers; SQLite can be replaced by PostgreSQL for more users or branches. *Verified by:* design review.
- **NFR6 Maintainability:** Modular design, readable code, configuration files and Git so a rule can change without rewriting other code. *Verified by:* code review.
- **NFR7 Availability:** Run locally with regular backups and a paper fallback during power or internet failure. *Verified by:* backup schedule check.
- **NFR8 Privacy:** Patient and prescription data limited to what dispensing needs and protected from unauthorised viewing. *Verified by:* access and data review.

4. Actors

Primary actors start interactions to achieve a goal; secondary actors support or receive results from the system.

- **Pharmacist** (primary actor): Records prescriptions, dispenses medicines and approves prescription-only sales.
- **Pharmacy assistant** (primary actor): Enters stock, searches batches, processes sales and returns.
- **Manager / owner** (primary actor): Manages suppliers and purchase orders, sets thresholds, reviews alerts and reports.
- **Administrator** (primary actor): Manages staff accounts, roles, backups and the audit log.
- **Supplier** (secondary actor): Delivers stock and receives purchase orders.
- **Prescriber (doctor)** (secondary actor): Issues the prescription that the pharmacist records.
- **Rules engine (system)** (secondary actor): Checks quantities and expiry dates after every change and creates alerts.

5. Use Cases

- **UC1 Login and manage users** - primary actor: Administrator; secondary actor: none; requirements: FR1, FR2.
- **UC2 Register medicine and add batch** - primary actor: Pharmacy assistant, Pharmacist; secondary actor: Supplier; requirements: FR3, FR4.
- **UC3 Search medicine and view batches** - primary actor: Pharmacist, Pharmacy assistant; secondary actor: none; requirements: FR5.
- **UC4 Record prescription** - primary actor: Pharmacist; secondary actor: Prescriber; requirements: FR6, FR7.
- **UC5 Dispense medicine / record sale** - primary actor: Pharmacist, Pharmacy assistant; secondary actor: none; requirements: FR8, FR9.
- **UC6 Record purchase, return or write-off** - primary actor: Pharmacy assistant; secondary actor: Supplier; requirements: FR8.
- **UC7 Manage suppliers** - primary actor: Manager / owner; secondary actor: Supplier; requirements: FR10.
- **UC8 Create purchase order and receive stock** - primary actor: Manager / owner; secondary actor: Supplier; requirements: FR11.
- **UC9 View low-stock alerts** - primary actor: Manager / owner; secondary actor: Rules engine; requirements: FR12.
- **UC10 View near-expiry and expired alerts** - primary actor: Manager / owner; secondary actor: Rules engine; requirements: FR13.
- **UC11 Generate reports** - primary actor: Manager / owner; secondary actor: none; requirements: FR15.
- **UC12 Configure reorder levels and warning period** - primary actor: Manager / owner; secondary actor: none; requirements: FR14.
- **UC13 Back up data and view audit log** - primary actor: Administrator; secondary actor: none; requirements: FR16.

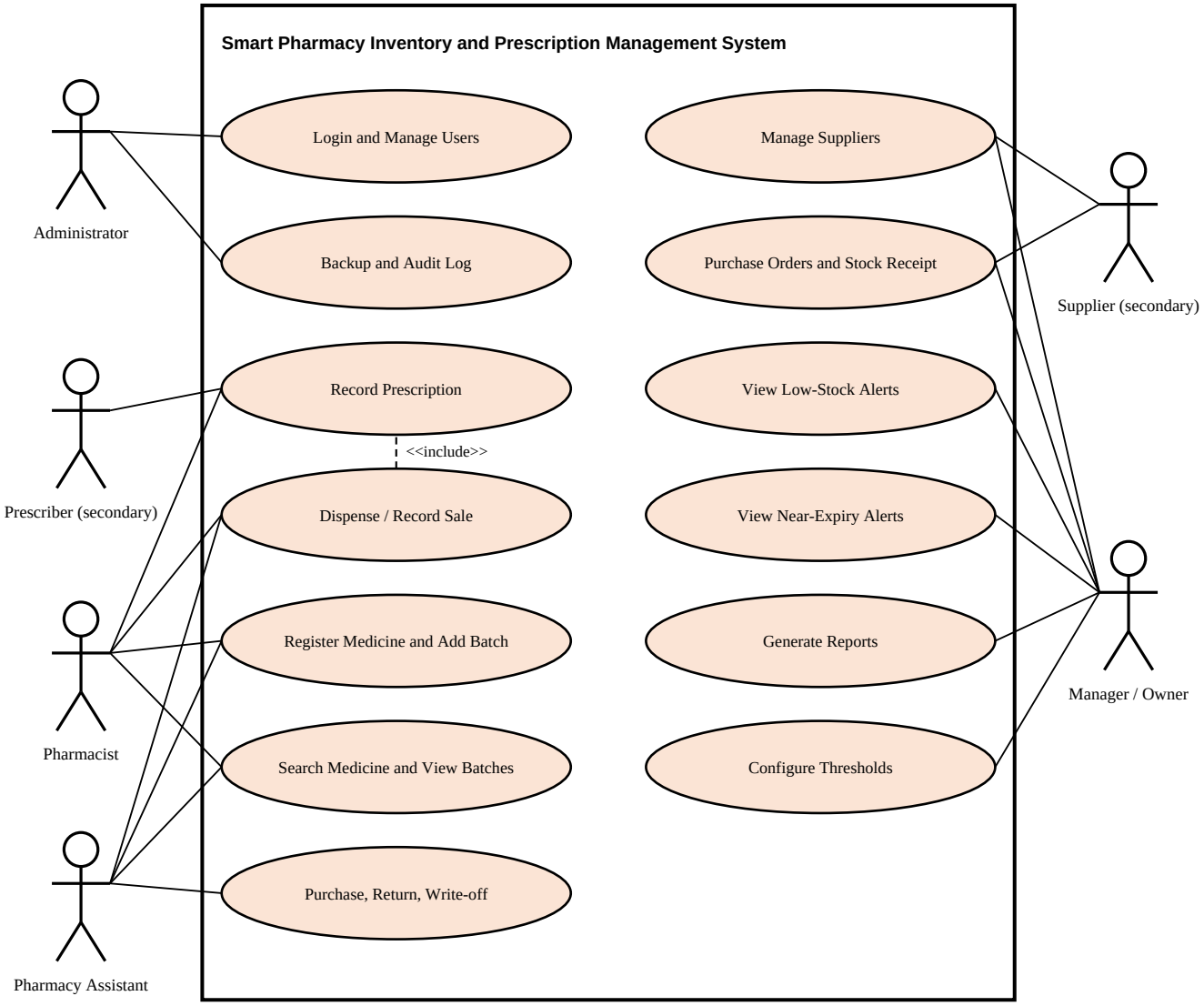
5.1 Use Case Descriptions

UC2 - Register medicine and add batch
Actor / precondition: Pharmacy assistant; user is logged in.
Main flow: 1 Open the medicine form and enter name, category, unit and reorder level. 2 System saves the medicine. 3 Assistant adds a batch with batch number, future expiry date, quantity and supplier. 4 System saves the batch and increases total stock.
Alternate / exception: Duplicate medicine, past expiry date, or quantity of 0 or less: entry rejected with a message and stock unchanged. Form submitted twice: only one record stored.
Postcondition: Medicine and batch stored; total stock updated; action recorded in the audit log.

UC4 - Record prescription
Actor / precondition: Pharmacist; logged in; prescription presented by the patient.
Main flow: 1 Pharmacist enters patient, prescriber, date and prescribed items. 2 System validates required fields. 3 System saves the prescription and makes it available to link to a sale.
Alternate / exception: Missing prescriber or item details: record rejected and the missing fields are shown.
Postcondition: Prescription stored and ready to be linked when an Rx-only medicine is dispensed.

6. Use Case Diagram

The diagram shows the actors outside the system boundary and the use cases UC1 to UC13 inside it. Dispense / Record Sale includes Record Prescription for prescription-only medicines. The Rules engine acts internally and is not drawn as an actor.



UC5 - Dispense medicine / record sale
Actor / precondition: Pharmacist (with assistant); logged in; medicine has valid unexpired stock.
Main flow: 1 Search the medicine. 2 System lists valid batches, earliest expiry first. 3 Enter quantity and the linked prescription if the medicine is prescription-only. 4 Pharmacist approves. 5 System saves the transaction and updates stock.
Alternate / exception: Expired batch or quantity above stock: sale blocked with the reason. Rx-only without prescription: denied. Duplicate submit: one record only.
Postcondition: Stock reduced; transaction logged with user, time and batch; low-stock alert raised once if the reorder level is reached.

6.1 Use Case Descriptions (continued)

UC1 - Login and manage users
Actor / precondition: Administrator (account management); any staff member (login). Account exists.
Main flow: 1 User enters credentials. 2 System checks the hashed password and role. 3 System opens the dashboard for that role. 4 Administrator can create, edit or deactivate accounts and assign roles.
Alternate / exception: Wrong password: access denied with a generic message; after 5 failures the account is locked for 10 minutes.
Postcondition: Authorised session started; login and account changes are written to the audit log.

UC8 - Create purchase order and receive stock
Actor / precondition: Manager / owner; supplier (secondary). Supplier and medicines are registered.
Main flow: 1 Manager selects a supplier and medicines to reorder. 2 System saves the purchase order. 3 On delivery, assistant checks batch number, expiry date and quantity. 4 System links the received batch to the order and increases stock.
Alternate / exception: Delivered quantity or batch differs from the order: mismatch is flagged for the manager. Expired or past-expiry delivery: rejected.
Postcondition: Purchase order closed or partly received; stock and transaction history updated.

UC9 - View low-stock alerts
Actor / precondition: Manager / owner; rules engine (secondary). Reorder level is configured for the medicine.
Main flow: 1 After every stock change the rules engine adds the valid quantity of all batches. 2 If the total is at or below the reorder level, it raises one alert. 3 Manager opens the dashboard and sees the medicine, quantity and usual supplier.
Alternate / exception: Quantity still above the level: no alert. Alert already raised: no duplicate alert until stock is replenished.
Postcondition: Alert visible within 60 s of the saved change; the system recommends a reorder but does not place it automatically.

UC10 - View near-expiry and expired alerts
Actor / precondition: Manager / owner; rules engine. Batches have expiry dates.
Main flow: 1 Rules engine compares each batch expiry date with today. 2 A batch within the warning period is marked near expiry and listed first for dispensing. 3 A batch past its date is marked expired and excluded from available stock.
Alternate / exception: Missing or invalid expiry date: batch is flagged for review instead of being treated as safe.
Postcondition: Alert names the medicine, batch and suggested action (for example: "Batch B1204 expires in 30 days").

UC12 - Configure reorder levels and warning period
Actor / precondition: Manager / owner; logged in with manager role.
Main flow: 1 Manager selects a medicine. 2 Enters a reorder level and near-expiry warning period. 3 System validates the values and saves them for that medicine.
Alternate / exception: Negative or non-numeric value: rejected. Assistant attempts the change: access denied and logged.
Postcondition: New thresholds apply to all later alert checks.

7. Software Requirement Specification (SRS)

7.1 Introduction

Purpose: to specify the requirements of the Smart Pharmacy Inventory and Prescription Management System for the development team, pharmacist, manager and faculty. **Product:** a web-based platform (Python Flask backend, SQLite database, rules engine, report generator) for one pharmacy. **Definitions:** a batch is a group of units of one medicine sharing one batch number and expiry date; FEFO means first-expiry-first-out; Rx-only means prescription-only.

7.2 Problem Description

Manual and paper-based stock handling lets batches expire unnoticed, causes stock-outs and permits prescription-only sales without a record. The system must record every medicine batch, link dispensing to prescriptions, manage suppliers and sales, and raise timely alerts.

7.3 Scope of the System

In scope: one pharmacy; registered medicines, batches, suppliers, users and prescriptions; web interface; rule-based alerts; downloadable reports. **Out of scope:** online ordering, insurance claims, e-prescription exchange with hospitals, automatic drug-interaction checking and fully automatic reordering.

7.4 Data Requirements

- **Medicine:** ID, name, category, unit, reorder level, Rx-only flag. One medicine has many batches.
- **Batch:** Batch number, expiry date, quantity, purchase date. Unique batch number per medicine; quantity 0 or more; expiry mandatory.
- **Supplier:** ID, name, contact, supplied medicines. Supplies many medicines; receives purchase orders.
- **Prescription:** ID, patient, prescriber, date, items. Linked to dispensed items; minimal patient data.
- **Transaction:** ID, type (sale, purchase, return, write-off), date, time, user, batch, quantity. Updates batch stock; one record per submission.
- **User / Audit log:** User ID, role, hashed password; change history. Role decides permitted actions.

7.5 External Interface Requirements

- **User interface:** Web pages usable on a computer, tablet or phone with clear alerts and quick search.
- **Hardware:** Optional barcode scanner to speed up stock entry and sales.
- **Software:** Flask backend and SQLite database; report export to PDF and CSV.
- **Communication:** Encrypted HTTPS connection between browser and server.

7.6 System Features

- **Medicine and batch management:** Medicines, batches, quantities, expiry dates and search (FR3 - FR5).
- **Prescription control:** Record prescriptions; require one for Rx-only medicines (FR6, FR7).
- **Sales and transactions:** Sales, purchases, returns, write-offs with FEFO and automatic stock update (FR8, FR9).
- **Supplier management:** Suppliers and purchase orders (FR10, FR11).
- **Alerts and reports:** Low-stock, near-expiry and expired alerts; exported reports (FR12 - FR15).

7.7 Data Dictionary

- **Medicine:** medicine_id (Integer; primary key, auto-generated); name, category, unit (Text; required; name unique); reorder_level (Integer; 0 or more; set per medicine); rx_required (Yes / No; marks prescription-only medicines).
- **Batch:** batch_id, batch_number (Integer, Text; batch number unique per medicine); expiry_date (Date; required; not in the past when added); quantity (Integer; 0 or more).
- **Supplier:** supplier_id, name, contact (Integer, Text; name and contact required).
- **Prescription:** prescription_id, patient, prescriber, date (Integer, Text, Date; all required for a valid record).
- **Transaction:** type, date_time, user_id, batch_id, quantity (Text, Timestamp, Integer; quantity above 0; one record per submission).
- **Purchase order:** order_id, supplier_id, status (Integer, Text; linked to received batches).
- **User:** user_id, role, password_hash (Integer, Text; role from a fixed list; password stored only hashed).
- **Audit log:** user_id, action, time (Integer, Text, Timestamp; cannot be edited by staff).

7.8 Business Rules

- **BR1** Low-stock alert: when the total valid quantity across a medicine's batches is at or below its reorder level (for illustration, 50 units). (source: FR12)
- **BR2** Near-expiry: a batch is near expiry when its expiry date is within the warning period (for illustration, 90 days). (source: FR13)
- **BR3** FEFO: the valid batch with the earliest expiry date is suggested first for dispensing. (source: FR9)
- **BR4** An expired batch is excluded from available stock and cannot be sold. (source: FR9, FR13)
- **BR5** A prescription-only medicine can be dispensed only with a linked prescription approved by a pharmacist. (source: FR7)
- **BR6** Negative, zero or impossible quantities and past expiry dates on new batches are rejected. (source: FR16)
- **BR7** The same submitted transaction is stored once, even if sent twice. (source: NFR3)
- **BR8** Only managers change thresholds; only administrators manage accounts. (source: NFR2)

Note: the values 50 units and 90 days are illustrations. The pharmacist or manager sets real values for each medicine according to demand, supplier lead time and storage needs.

8. Assumptions and Constraints

8.1 Assumptions

- Staff have basic computer skills and use a computer, tablet or phone with a browser.
- The first version serves one pharmacy with one database; barcodes are optional.
- Reorder levels (for example 50 units) and the near-expiry period (for example 90 days) are set by the pharmacist or manager for each medicine.
- Every purchase, sale and return is entered into the system so that stock figures stay current.
- The pharmacist makes every clinical decision; the system only records and alerts.

8.2 Constraints

- **Technical:** Prototype built with Flask and SQLite; single pharmacy; no automatic drug-interaction checking.
- **Operational:** Accuracy depends on correct data entry; physical stock must still be counted from time to time.
- **Economic:** Student team using free, open-source tools; no budget for specialised hardware.
- **Legal:** Patient and prescription data must be kept private and accessed only by authorised users.
- **Time:** Four one-week sprints: login and registration, batch tracking, prescriptions and alerts, reports and testing.

9. Validation of Completeness and Consistency

9.1 Traceability: objectives covered by requirements

- **Maintain medicine inventory; track batches and expiry** is covered by FR3, FR4, FR5, FR13 and use cases UC2, UC3, UC10.
- **Manage prescriptions** is covered by FR6, FR7 and use cases UC4, UC5.
- **Generate low-stock alerts** is covered by FR12, FR14 and use cases UC9, UC12.
- **Manage supplier information** is covered by FR10, FR11 and use cases UC7, UC8.
- **Track medicine transactions** is covered by FR8, FR9 and use cases UC5, UC6.
- **Identify near-expiry products; generate reports** is covered by FR13, FR15 and use cases UC10, UC11.

9.2 Quality checks and suggested improvements

- **Completeness:** All eight objectives and all stakeholders map to at least one requirement and use case. *Improvement:* keep the traceability table updated.
- **Consistency:** Blocking expired batches (FR9) agrees with alert rules (FR13); no conflicting rules found. *Improvement:* use one rules engine for all alerts.
- **Ambiguity:** Words such as "quickly" are replaced by targets of 3 s and 60 s. *Improvement:* give every non-functional requirement a measure.
- **Redundancy:** Low-stock and expiry alerts share one rule mechanism. *Improvement:* merge into one alert feature.
- **Missing:** Stock adjustment with a reason, backup and restore, and report scheduling were not explicit. *Improvement:* add as requirements and review with the pharmacist.

10. Requirement Verification and Conclusion

10.1 Verification matrix (test scenarios)

- **Enter a negative quantity, missing batch number or past expiry date** (requirements FR4, FR16): entry rejected or flagged; stock unchanged.
- **Sell more than the available quantity** (requirements FR8, FR9): sale blocked; stock unchanged.
- **Try to sell an expired batch** (requirements FR9, FR13): sale blocked and the reason shown.
- **Dispense an Rx-only medicine without a linked prescription** (requirements FR7): transaction denied.
- **Reduce stock to the reorder level** (requirements FR12): one low-stock alert, not before the level is reached.
- **Submit the same sale twice** (requirements NFR3): only one transaction stored.
- **Assistant tries to change a threshold** (requirements FR14, NFR2): access denied and logged.
- **Compare an exported report with stored records** (requirements FR15): totals match exactly.
- **Search a medicine in a prototype-sized catalogue** (requirements FR5, NFR1): batches shown within 3 s.

10.2 Requirement summary and priority

- **Functional requirements:** 16 - 13 Must, 3 Should (FR11, FR14, FR15).
- **Non-functional requirements:** 8 - Performance, security, reliability, usability, scalability, maintainability, availability, privacy.
- **Actors / use cases:** 7 / 13 - 4 primary human actors, 2 secondary human actors, 1 system actor.
- **Business rules / data entities:** 8 / 7 - Rules in section 7.8; entities in sections 7.4 and 7.7.

10.3 Requirement review checklist

- Each requirement is unique, testable and stated with "shall"; measurable targets replace vague words.
- Every stakeholder need and every objective maps to a requirement and a use case.
- No two requirements conflict; alert and blocking rules use the same expiry and quantity data.
- Requirements are reviewed with the pharmacist and manager before each sprint and updated after feedback.

10.4 Conclusion

The scenario-based analysis shows that the pharmacy needs batch-level stock records, prescription control, supplier and transaction tracking, and rule-based alerts. The specification lists 16 functional and 8 non-functional requirements, 7 actors, 13 use cases and 8 business rules, and checks them for completeness, consistency and ambiguity. The system supports the pharmacist by recording data and raising alerts, while every professional decision stays with the pharmacist. Its usefulness depends on accurate data entry and on the targets being confirmed in prototype testing.

11. Requirements Summary Table

The functional requirements are grouped into 8 features (FR1 to FR8). The last column shows the detailed requirements (FR1-FR16) covered in Section 2.

11.1 Functional Requirements (FR)

ID	Feature	Requirement	Covers
FR1	User login and roles	The system shall provide secure role-based login for pharmacist, assistant, manager and administrator, and let the administrator create, edit and deactivate accounts.	FR1, FR2
FR2	Medicine and batch records	The system shall register medicines (name, category, unit, storage, reorder level) and store each batch (batch number, expiry date, quantity, supplier, purchase date), with search by name or barcode showing all batches.	FR3, FR4, FR5
FR3	Prescription control	The system shall record prescriptions (patient, prescriber, date, items), flag prescription-only medicines and require a linked prescription before dispensing.	FR6, FR7
FR4	Sales and transactions	The system shall record sales, purchases, returns and write-offs, update stock automatically, suggest the earliest-expiring valid batch (FEFO) and block expired batches or quantity above stock.	FR8, FR9
FR5	Supplier and purchase orders	The system shall maintain supplier details and supplied medicines, and create purchase orders linked to received stock.	FR10, FR11
FR6	Alerts and thresholds	The system shall raise low-stock alerts (naming medicine and supplier), mark near-expiry and expired batches (excluding expired from stock), and let managers set reorder levels and the warning period per medicine.	FR12, FR13, FR14
FR7	Report export	The system shall export inventory, sales and expiry reports for a chosen period as PDF or CSV.	FR15
FR8	Audit log and validation	The system shall keep an audit log of who changed what and when, and reject negative or impossible quantities.	FR16

11.2 Non-Functional Requirements (NFR)

ID	Feature	Requirement
NFR1	Performance	Search a medicine and display its batches within 3 seconds; show a triggered alert within 60 seconds of a saved change.
NFR2	Security	Hashed passwords, role-based permissions, HTTPS and an audit log; only pharmacists approve Rx-only sales; only managers change thresholds.
NFR3	Reliability	Validate every entry, store exactly one record per submitted transaction, keep transaction history and back up regularly.
NFR4	Usability	Simple screens, clear units, readable alert messages (e.g. "Batch B1204 expires in 30 days"), search by name or barcode.
NFR5	Scalability	Every record carries pharmacy, medicine and batch identifiers; SQLite can be replaced by PostgreSQL for more users or branches.
NFR6	Maintainability	Modular design, readable code, configuration files and Git so a rule can change without rewriting other code.
NFR7	Availability	Run locally with regular backups and a paper fallback during power or internet failure.
NFR8	Privacy	Patient and prescription data limited to what dispensing needs and protected from unauthorised viewing.